

OmniClaw

Economic Execution and Control Layer for Agentic Systems

Omnuron AI

omniclaw.ai

Version 1.0 · March 2026

Abstract

Autonomous software agents are becoming independent economic actors, transacting at machine speed across heterogeneous payment rails. Existing wallet and settlement infrastructure provides transaction primitives but not the control model required for safe, governed autonomous execution. We present OmniClaw, an economic execution and control layer that unifies buyer-side and seller-side payment flows — across direct USDC transfers, HTTP 402 protocol payments, cross-chain CCTP settlement, and gasless Circle Gateway nanopayments — while enforcing programmable spending policy, identity-aware trust gating via ERC-8004 on-chain registries, concurrency-safe fund management through distributed two-phase locking, and settlement-aware state semantics. We formalize the execution pipeline, the guard reservation model, the Weighted Trust Score function, and the seller-side replay resistance properties. We prove that the system provides budget non-overcommitment under concurrent load, idempotent payment execution under arbitrary retries, and seller-side replay resistance under distributed deployment, subject to the linearizability of the backing storage system.

Keywords: autonomous agents, payment infrastructure, x402 protocol, ERC-8004, concurrency control, trust scoring, USDC, nanopayments, agentic commerce.

1. Introduction

The emergence of autonomous software agents capable of initiating, authorizing, and completing financial transactions without human mediation represents a structural change in how economic activity occurs on the internet. Agents are no longer passive tools that surface information or compose communications. They act: purchasing inference time, acquiring real-time data feeds, paying for compute, settling machine-to-machine service fees, and accepting revenue for services they expose to other agents.

This transition surfaces a class of correctness problems that existing payment infrastructure was not designed to address. Wallet APIs and settlement primitives provide the means to move funds but offer no model for governing *when* funds should move, *whether* a counterparty should be trusted, or *how* concurrent workers sharing a wallet avoid overcommitting capital. The result is that production agentic systems must reconstruct policy enforcement, concurrency controls, idempotency guarantees, and trust evaluation independently — and inconsistently — at the application layer.

OmniClaw is an application-layer payment control plane that addresses this gap by composition rather than replacement. It provides a unified execution surface across heterogeneous payment rails with deterministic pre-execution safety checks, distributed two-phase locking, explicit settlement-state semantics, and identity-aware trust gating derived from on-chain ERC-8004 registries.

The remainder of this paper is organized as follows. Section 2 surveys the infrastructure OmniClaw composes. Section 3 formalizes the failure classes. Section 4 presents the protocol design and formal execution model. Section 5 states the security theorems. Section 6 examines the economic structure of the agentic commerce layer. Section 7 discusses open problems and future work.

2. Background

2.1 The x402 Payment Protocol

The HTTP 402 status code was reserved in RFC 2616 [9] as "Payment Required" but left without a standard definition for over three decades. The x402 protocol, standardized by the x402 Foundation in 2025 [3,4], fills this gap. A protected resource responds to an unpaid request with a 402 Payment Required status containing machine-readable requirements: price, accepted payment schemes, network identifier, and recipient address. The client constructs a cryptographically signed payment payload and retries. An external facilitator verifies and settles the proof; the server grants access upon confirmation. x402 v2 separates the negotiation protocol from payment scheme, enabling pluggable settlement backends.

2.2 ERC-8004: Agent Identity and Reputation

ERC-8004 [5] defines three interoperable on-chain registries for agent coordination. The Identity Registry (ERC-721-based) records agent registrations. The Reputation Registry aggregates weighted feedback signals from registered agents. The Validation Registry holds attestation-based trust claims. Together they provide a verifiable signal layer for assessing counterparty identity and reputation before any economic interaction.

2.3 Circle Gateway and Nanopayments

Circle Gateway [7] provides chain-abstracted USDC balances and cross-chain settlement. Circle Nanopayments [6], built on Gateway, enables gas-free USDC transfers as small as \$0.000001 by aggregating EIP-3009 [8] signed authorizations off-chain and settling net positions in batched on-chain transactions. The per-transaction gas cost approaches zero as batch size grows. Settlement executes inside an AWS Nitro Enclave TEE, preserving non-custodial properties while eliminating per-payment blockchain overhead.

3. Problem Formulation

Let W be a set of agent wallets with balance $B(w, t) \in \mathbf{R}_{\geq 0}$ at time t . A payment request is a tuple (w, r, a) where $w \in W$, r is the recipient, and $a > 0$ is the amount. We identify six failure classes arising without a governing control layer.

3.1 Budget Overcommitment Under Concurrency

When n concurrent workers each read $B(w, t)$ and each authorize a payment of amount a_i , total authorized spend may exceed $B(w, t)$ if checks are not serialized. The correctness condition:

$$\sum_{i=1..n} a_i \leq B(w, t)$$

may hold for each worker individually at read time but fail in aggregate at commit time, because $B(w, t)$ is not updated between concurrent reads.

3.2 Non-Idempotent Execution Under Retries

A payment function $f(w, r, a)$ is *idempotent* if repeated application with identical arguments produces exactly one charge. Without a deterministic idempotency key derived from payment parameters, k retry attempts produce k charges:

$$\sum_{j=1..k} f(w, r, a) = k \cdot a \quad \text{instead of the required } a$$

3.3 Seller-Side Replay Attacks

A payment proof $\pi = (\text{network}, \text{payer}, \text{payee}, \text{contract}, \text{nonce}, \text{signature})$ is valid if the EIP-3009 signature verifies. A seller without a consumed-nonce registry N accepts any valid proof multiple times, granting the resource indefinitely from a single payment authorization.

3.4 False Settlement Finality

Let terminal states $T = \{\text{SETTLED}, \text{FAILED_FINAL}, \text{CANCELLED}, \text{BLOCKED}\}$ and in-flight states $I = \{\text{AUTHORIZED}, \text{PENDING}, \text{PROCESSING}, \text{PENDING_SETTLEMENT}\}$. A system that maps any state in I to a success response conflates *accepted* with *settled*, creating reconciliation risk.

3.5 Trust-Blind Counterparty Exposure

Without an evaluation function $T(r) \rightarrow \{\text{APPROVED}, \text{HELD}, \text{BLOCKED}\}$, an agent must either accept all recipients — including fraudulent ones — or reject all unknown recipients. Neither is acceptable for production autonomous systems.

3.6 Policy Bypass Under Fragmented Enforcement

Spending policy defined in application code is enforced independently at each call site. Under concurrency, evaluation against stale state permits policy violations. Correct enforcement requires atomic evaluation against shared reservation state.

4. Protocol Design

OmniClaw resolves each failure class through four composable subsystems: a guard and reservation system, a distributed fund-lock protocol, an identity-aware trust gate, and a seller-side nonce registry. We present each formally.

4.1 Formal Definitions

Definition 1 (Available Balance)

Let $R(w, t)$ denote total reserved spend capacity for wallet w at time t , summed across all active reservations:

$$R(w, t) = \sum_{i \in \text{active}(w, t)} \text{amount}(i)$$

The available balance is:

$$\text{avail}(w, t) = B(w, t) - R(w, t)$$

A payment of amount a is authorized only if $\text{avail}(w, t) \geq a$, evaluated within the exclusive fund-lock critical section.

Definition 2 (Idempotency Key)

For a payment request $(w, r, a, \text{purpose})$, the idempotency key is:

$$k = H(w \parallel r \parallel a \parallel \text{purpose})$$

where H is a collision-resistant hash function and $\parallel$ denotes concatenation. Requests sharing key k within window δ are deduplicated without re-execution.

Definition 3 (Weighted Trust Score)

Let $F(r) = \{f_1, \dots, f_n\}$ be reputation feedback signals for r , each $f_i = (\text{score}_i, \text{age}_i, \text{verified}_i)$. Define:

$$\delta(\text{age}) = \max(0.2, 1.0 - 0.5 \cdot \log_2(1 + \text{age}/30))$$

$$\lambda(\text{verified}) = 1.5 \text{ if } \text{verified} = \text{true}, \text{ else } 1.0$$

Excluding self-submissions, the WTS for r is:

$$\text{WTS}(r) = \sum_i \text{score}_i \cdot \delta(\text{age}_i) \cdot \lambda(\text{verified}_i) / \sum_i \delta(\text{age}_i) \cdot \lambda(\text{verified}_i)$$

If $|F(r)| < 3$, r is classified as a new agent regardless of score.

Definition 4 (Nonce Replay Registry)

The nonce registry N is a persistent set of consumed proofs. A proof $\pi = (\text{net}, \text{payer}, \text{payee}, \text{contract}, \text{nonce}, \text{sig})$ is admitted if and only if:

$$\pi \notin N \wedge \text{verify}(\text{sig}, (\text{net}, \text{payer}, \text{payee}, \text{contract}, \text{nonce})) = \text{true}$$

On admission, $N := N \cup \{\pi\}$. In distributed deployments N must be backed by a linearizable store.

4.2 Buyer-Side Execution Pipeline

Every outbound payment executes the following deterministic ten-step pipeline. Failure at any step halts execution and triggers guard rollback and ledger update.

1. **Input validation and idempotency.** Derive key k per Definition 2. If $k \in \text{dedup window}$, return the cached result without re-execution.
2. **Trust Gate evaluation.** Compute $\text{WTS}(r)$ per Definition 3 and evaluate the 10-check policy. Reject if verdict = BLOCKED; queue for review if verdict = HELD.
3. **Ledger entry creation.** Create an audit record before any funds move. This record persists regardless of outcome.
4. **Guard chain evaluation.** For each guard g_i with limit L_i and consumption $C_i(w, t)$, the guard passes if $C_i(w, t) + a \leq L_i$. On pass, reserve a against each guard. On any failure, release all reservations for this attempt.
5. **Fund lock acquisition.** Acquire distributed mutex $\mu(w)$, serializing the critical section (Steps 6–8) across all concurrent workers.
6. **Available balance verification.** Within the lock, verify $\text{avail}(w, t) \geq a$ per Definition 1. Deny if the condition fails.
7. **Adapter dispatch.** Select the protocol adapter: direct transfer, x402, CCTP cross-chain, or Gateway nanopayment, based on recipient type, network, and amount threshold.
8. **Execution with resilience.** Execute the adapter under a circuit-breaker and retry shell with exponential backoff on transient failures.
9. **Guard commit or rollback.** On success: $C_i(w, t) := C_i(w, t) + a$. On failure: release all reservations.
10. **Fund lock release.** Release $\mu(w)$ unconditionally.

4.3 Guard System

Guards are composable predicates over payment request and wallet state. They evaluate as a conjunction — all guards must pass. Guard state transitions use atomic compare-and-swap operations.

Guard	Predicate (must hold to pass)	Reserve on pass
BudgetGuard	$C_daily(w,t) + a \leq L_daily \wedge C_hourly(w,t) + a \leq L_hourly$	a against each window
RateLimitGuard	$count_min(w,t) < R_min \wedge count_hour(w,t) < R_hour$	1 against each counter
SingleTxGuard	$L_min \leq a \leq L_max$	none (evaluation only)
RecipientGuard	$r \in whitelist(w) \text{ or } r \notin blacklist(w)$	none (evaluation only)
ConfirmGuard	$a < threshold(w) \text{ or } approved(intent)$	hold if $a \geq threshold$

4.4 Payment Intent Protocol

Payment intents provide a two-phase execution model. An intent $I = (w, r, a, \text{purpose}, \text{state})$ traverses:

```

REQUIRES_CONFIRMATION → {PROCESSING | CANCELLED}
REQUIRES_REVIEW → {PROCESSING | CANCELLED}
PROCESSING → {SUCCEEDED | FAILED}

```

Creation evaluates the guard chain and reserves $R(w) += a$ without executing payment. Confirmation executes Steps 5–10 within a fresh lock scope. Cancellation releases the reservation with no charge. Trust-held intents require operator approval before confirmation is permitted.

4.5 Seller-Side Acceptance Protocol

The seller middleware protects any HTTP endpoint. Let π be the payment proof and req the declared requirements.

- Unpaid request.** Return 402 with requirements per x402 v2, including the GatewayWalletBatched scheme identifier and verifying contract address for EIP-3009 domain construction.
- Buyer authorization.** Buyer signs an EIP-3009 TransferWithAuthorization off-chain. Validity window must satisfy $validBefore \geq now + 3 \text{ days}$.
- Signed retry.** Buyer retries with the base64-encoded proof π as the PAYMENT-SIGNATURE header.

4. **Nonce validation.** Verify $\pi \notin N$ per Definition 4. A consumed nonce causes immediate 402 rejection.
5. **Facilitator settlement.** Call Circle Gateway `settle( $\pi$ , req)`. Gateway verifies the signature in the TEE, locks buyer balance, and credits seller. `settle()` is called directly — not `verify()` then `settle()` — for atomic, low-latency settlement.
6. **Resource delivery.** Admit π into N and serve the resource with `PAYMENT-RESPONSE` header. Gateway batches pending authorizations and settles net positions on-chain.

4.6 Settlement State Semantics

Define the payment state alphabet $\Sigma = I \cup T$ where $I = \{\text{PENDING, AUTHORIZED, PROCESSING, PENDING_SETTLEMENT}\}$ and $T = \{\text{SETTLED, FAILED_FINAL, CANCELLED, BLOCKED}\}$. Strict settlement mode enforces:

$$\text{success}(p) \Leftrightarrow \text{state}(p) = \text{SETTLED}$$

No state in I may be reported as a terminal success. Systems requiring settlement guarantees must subscribe to ledger events until $\text{state}(p) \in T$.

5. Security Properties

We state six security theorems and sketch the proof for each. All proofs assume a linearizable distributed storage backend; properties degrade proportionally if this assumption is violated.

Theorem 1 (Budget Non-Overcommitment)

For any set of concurrent requests $\{(w, r_i, a_i)\}$ against wallet w , committed spend never exceeds the configured limit L :

$$\sum_{i:\text{committed}(i)} a_i \leq L$$

Proof: guard reservation is atomic against shared state before fund-lock acquisition. Available balance is re-verified within the exclusive lock scope. Overcommitment requires a linearizability violation in the reservation backend.

Theorem 2 (Idempotent Execution)

For any request $(w, r, a, \text{purpose})$ submitted $k \geq 1$ times within deduplication window δ , exactly one charge is applied:

$$\text{charges}(w, r, a, \text{purpose}, \delta) = 1$$

Proof: key $k = H(w||r||a||\text{purpose})$ is deterministic. On first execution k is admitted to the dedup store. Subsequent submissions within δ return the cached result without re-execution.

Theorem 3 (Seller Replay Resistance)

A payment proof π authorizes at most one resource delivery:

$$\text{deliveries}(\pi) \leq 1$$

Proof: on first admission π is inserted into N . Definition 4 requires $\pi \notin N$ for admission. A linearizable store ensures no two concurrent checks can both observe $\pi \notin N$ simultaneously. In-memory storage is insufficient for multi-instance deployments.

Theorem 4 (Settlement Finality Integrity)

The system reports success for p if and only if $\text{state}(p) = \text{SETTLED}$:

$$\text{report_success}(p) \Leftrightarrow \text{state}(p) = \text{SETTLED}$$

Proof: follows directly from the strict settlement invariant in Section 4.6. The state machine is monotone: no transition from SETTLED back into I is permitted.

Theorem 5 (Trust Gate Non-Bypass)

A payment to r executes only if $T(r) = \text{APPROVED}$:

$$\text{executed}(w, r, a) \Rightarrow T(r) = \text{APPROVED}$$

Proof: Trust Gate evaluation (Step 2) precedes guard reservation (Step 4), fund-lock acquisition (Step 5), and adapter dispatch (Step 7). Rejection at Step 2 halts the pipeline before any of these steps execute.

Theorem 6 (Webhook Integrity)

A notification msg is processed if and only if it carries a valid Ed25519 signature from verification key K and its ID n is fresh:

$$\text{process}(\text{msg}) \Leftrightarrow \text{verify}_K(\text{sig}, \text{msg}) = \text{true} \wedge n \notin N_{\text{wh}}$$

On admission: $N_{\text{wh}} := N_{\text{wh}} \cup \{n\}$.

5.1 Residual Risks

Three risk categories exist outside the protocol boundary.

- **Provider non-idempotency.** If the downstream custody or Gateway API lacks idempotency, duplicate charges may occur despite OmniClaw's deduplication. Operators should verify provider-level guarantees independently.
- **Storage non-linearizability.** Theorems 1, 3, and 6 require a linearizable backend. Eventually-consistent or misconfigured stores degrade these properties in proportion to their consistency violations.
- **Sparse identity coverage.** WTS is effective only for agents with $|F(r)| \geq 3$. New agents trigger the new-agent heuristic regardless of actual behaviour.

6. The Agentic Economy Layer

6.1 Stack Positioning

The emerging agentic economy stratifies into five layers: cloud infrastructure; agent frameworks providing orchestration and memory; payment and trust infrastructure enabling machine-to-machine transactions; autonomous agents operating as economic actors; and AGaaS (Agentic-as-a-Service) products delivering closed-loop outcomes to human customers. OmniClaw occupies the third layer — the missing foundation that makes the upper two viable at scale.

6.2 Two-Sided Network Dynamics

OmniClaw is explicitly symmetric. Every participant is simultaneously a potential buyer and seller. Let $A(t)$ denote the set of registered agents at time t . The utility of the trust layer for a given payment is a monotone increasing function of $|A(t)|$: more registrations reduce the fraction of counterparties subject to the new-agent heuristic and increase WTS signal density. Identity value therefore accumulates through registration alone, independent of transaction volume — a compounding network effect.

6.3 Economics of Batched Settlement

Define per-payment gas cost under individual on-chain settlement as g . For a batch of n payments settled at total cost G , the effective per-payment cost is:

$$g_{\text{eff}}(n) = G / n \rightarrow 0 \text{ as } n \rightarrow \infty$$

This eliminates the fixed-cost floor that makes sub-cent transactions uneconomical under individual settlement. Billing models previously infeasible at agent frequencies — per-API-call charges of \$0.001, per-second compute pricing, per-inference micropayments — become viable when $g_{\text{eff}}(n)$ approaches zero.

6.4 Enterprise Governance Alignment

OmniClaw's formal properties map directly to enterprise governance requirements. Budget guards satisfy treasury controls. The HITL ConfirmGuard satisfies approval requirements for high-value transactions. Idempotency guarantees satisfy reconciliation requirements. Strict settlement semantics satisfy finality reporting requirements. The Trust Gate satisfies counterparty verification requirements. Together, these properties make agentic payments governable as enterprise infrastructure rather than uncontrolled financial exposure.

7. Discussion and Future Work

7.1 Open Problems

Formal policy composition. The guard chain currently evaluates as a strict conjunction. A richer policy algebra allowing disjunction, temporal constraints, and cross-wallet aggregation would expand the expressible policy space beyond per-guard independent predicates.

Trust registry decentralisation. ERC-8004 identity data originates from a limited set of deployed contracts. An operator-hosted canonical registry with a public query API would accelerate $|A(t)|$ growth and the corresponding utility of WTS(r) across the ecosystem.

Cross-facilitator interoperability. The current protocol couples to Circle Gateway as the canonical facilitator. A generalised facilitator interface would introduce competition at the settlement layer, reducing single-provider dependency.

Streaming value and deferred settlement. The x402 v2 specification supports deferred payment schemes that aggregate authorizations and settle periodically [4]. Integrating deferred scheme support would enable continuous value flows at per-second or per-unit granularity.

Formal verification. The proofs in Section 5 are sketches. Formal verification using model checking or interactive theorem proving — particularly of Theorem 1 under realistic concurrent execution models — would strengthen the correctness guarantees beyond sketch level.

7.2 Limitations

OmniClaw is an application-layer control plane, not a settlement network. Its correctness properties are conditioned on the correctness of the external providers it composes: custody APIs, chain RPC nodes, and the Circle Gateway facilitator. Provider-side failures or behavioral changes propagate to the execution plane regardless of OmniClaw's internal correctness. The system cannot provide settlement guarantees stronger than those of its underlying facilitator.

7.3 Conclusion

Autonomous commerce requires deterministic economic control, not only transaction primitives. The failure modes of uncontrolled agentic payment access — overcommitment, duplicate charges, replay exploitation, false finality, trust-blind exposure — are predictable consequences of deploying payment APIs without a governing control layer above them.

OmniClaw provides this layer through a formally specified execution pipeline, a composable guard and reservation model, a cryptographically grounded trust scoring function, and a nonce-registry-backed seller acceptance protocol. The six theorems in Section 5 characterize the

guarantees the system provides and the conditions under which each holds.

As autonomous agents become primary economic actors — purchasing compute, selling inference, licensing data, and settling machine-to-machine service fees at scale — the infrastructure governing their financial behavior becomes as critical as the infrastructure governing their reasoning. OmniClaw is that infrastructure.

References

- [1] S. Nakamoto. "Bitcoin: A Peer-to-Peer Electronic Cash System." 2008. <https://bitcoin.org/bitcoin.pdf>
- [2] Bittensor Foundation. "Bittensor: A Peer-to-Peer Intelligence Market." 2022. <https://bittensor.com/whitepaper>
- [3] Coinbase Developer Platform. "x402: An Open Payment Standard for the Internet." 2025. <https://x402.org>
- [4] x402 Foundation. "x402 Protocol Specification v2." 2025. <https://github.com/coinbase/x402>
- [5] Ethereum Improvement Proposals. "ERC-8004: Trustless Agents Standard." <https://eips.ethereum.org/EIPS/eip-8004>
- [6] Circle. "Nanopayments: Gas-Free USDC Transfers for the Agentic Economy." 2026. <https://developers.circle.com/gateway/nanopayments>
- [7] Circle. "Circle Gateway: Chain-Abstracted USDC Infrastructure." 2025. <https://developers.circle.com/gateway>
- [8] P. Castonguay, D. Kim. "EIP-3009: Transfer With Authorization." EIP. 2020. <https://eips.ethereum.org/EIPS/eip-3009>
- [9] R. Fielding et al. "Hypertext Transfer Protocol — HTTP/1.1." RFC 2616. IETF. 1999.
- [10] P. Bernstein, V. Hadzilacos, N. Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley. 1987.
- [11] M. Fischer, N. Lynch, M. Paterson. "Impossibility of Distributed Consensus with One Faulty Process." *Journal of the ACM*, 32(2):374-382. 1985.
- [12] L. Lamport. "The Part-Time Parliament." *ACM Transactions on Computer Systems*, 16(2):133-169. 1998.
- [13] J. Gray, A. Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann. 1992.
- [14] C. Dwork, N. Lynch, L. Stockmeyer. "Consensus in the Presence of Partial Synchrony." *Journal of the ACM*, 35(2):288-323. 1988.